

Strings (Intermediate)

1. Score of a String

You are given a string s . The score of a string is defined as the sum of the absolute difference

between the ASCII values of adjacent characters.

Return the score of s .

Example 1:

Input: $s = \text{"hello"}$

Output: 13

Explanation:

The ASCII values of the characters in s are:

'h' = 104, 'e' = 101, 'l' = 108, 'o' = 111.

So, the score of s would be

$|104 - 101| + |101 - 108| + |108 - 108| + |108 - 111| = 3 + 7 + 0 + 3 = 13$.

Example 2:

Input: $s = \text{"zaz"}$

Output: 50

Explanation:

The ASCII values of the characters in s are:

'z' = 122, 'a' = 97

So, the score of s would be

$|122 - 97| + |97 - 122| = 25 + 25 = 50$.

Constraints:

- $2 \leq s.length \leq 100$
- s consists only of lowercase English letters.

2. Minimum Number of Operations to Move All Balls to Each

Box

You have n boxes. You are given a binary string $boxes$ of length n , where $boxes[i]$ is '0' if the i th

box is empty, and '1' if it contains one ball.

In one operation, you can move one ball from a box to an adjacent box. Box i is adjacent to box j if

$\text{abs}(i - j) == 1$. Note that after doing so, there may be more than one ball in some boxes.

Return an array $answer$ of size n , where $answer[i]$ is the minimum number of operations needed to move all the balls to the i th box.

Each `answer[i]` is calculated considering the initial state of the boxes.

Example 1:

Input: `boxes = "110"`

Output: `[1,1,3]`

Explanation:

The answer for each box is as follows:

1. First box: you will have to move one ball from the second box to the first box in one operation.
2. Second box: you will have to move one ball from the first box to the second box in one operation.
3. Third box: you will have to move one ball from the first box to the third box in two operations, and move one ball from the second box to the third box in one operation.

Example 2:

Input: `boxes = "001011"`

Output: `[11,8,5,4,3,4]`

Constraints:

- `n == boxes.length`
- `1 <= n <= 2000`
- `boxes[i]` is either '0' or '1'.

3. Jewels and Stones

You're given strings `jewels` representing the types of stones that are jewels, and `stones` representing

the stones you have. Each character in `stones` is a type of stone you have. You want to know how

many of the stones you have are also jewels.

Letters are case sensitive, so "a" is considered a different type of stone from "A".

Example 1:

Input: `jewels = "aA"`, `stones = "aAAbbbb"`

Output: 3

Example 2:

Input: `jewels = "z"`, `stones = "ZZ"`

Output: 0

Constraints:

- `1 <= jewels.length, stones.length <= 50`
- `jewels` and `stones` consist of only English letters.
- All the characters of `jewels` are unique.

4. Maximum Number of Words Found in Sentences

A sentence is a list of words that are separated by a single space with no leading or trailing spaces.

You are given an array of strings `sentences`, where each `sentences[i]` represents a single sentence.

Return the maximum number of words that appear in a single sentence.

Example 1:

Input: `sentences = ["alice and bob love leetcode", "i think so too", "this is great thanks very much"]`

Output: 6

Explanation:

- The first sentence, "alice and bob love leetcode", has 5 words in total.
- The second sentence, "i think so too", has 4 words in total.
- The third sentence, "this is great thanks very much", has 6 words in total.

Thus, the maximum number of words in a single sentence comes from the third sentence, which has 6 words.

Example 2:

Input: `sentences = ["please wait", "continue to fight", "continue to win"]`

Output: 3

Explanation: It is possible that multiple sentences contain the same number of words. In this example, the second and third sentences have the same number of words.

Constraints:

- $1 \leq \text{sentences.length} \leq 100$
- $1 \leq \text{sentences}[i].\text{length} \leq 100$
- `sentences[i]` consists only of lowercase English letters and ' ' only.
- `sentences[i]` does not have leading or trailing spaces.
- All the words in `sentences[i]` are separated by a single space.

5. Truncate Sentence

A sentence is a list of words that are separated by a single space with no leading or trailing spaces.

Each of the words consists of only uppercase and lowercase English letters (no punctuation).

For example, "Hello World", "HELLO", and "hello world hello world" are all sentences.

You are given a sentence **s** and an integer **k**. You want to truncate **s** such that it contains only the first **k** words. Return **s** after truncating it.

Example 1:

Input: **s** = "Hello how are you Contestant", **k** = 4

Output: "Hello how are you"

Explanation:

The words in **s** are ["Hello", "how", "are", "you", "Contestant"].

The first 4 words are ["Hello", "how", "are", "you"].

Hence, you should return "Hello how are you".

Example 2:

Input: **s** = "What is the solution to this problem", **k** = 4

Output: "What is the solution"

Explanation:

The words in **s** are ["What", "is", "the", "solution", "to", "this", "problem"].

The first 4 words are ["What", "is", "the", "solution"]

Example 3:

Input: **s** = "chopper is not a tanuki", **k** = 5

Output: "chopper is not a tanuki"

Constraints:

- $1 \leq s.length \leq 500$
- **k** is in the range [1, the number of words in **s**].
- **s** consists of only lowercase and uppercase English letters and spaces.
- The words in **s** are separated by a single space.
- There are no leading or trailing spaces.

6. Find First Palindromic String in the Array

Given an array of strings **words**, return the first palindromic string in the array. If there is no such string, return an empty string ""

.

A string is palindromic if it reads the same forward and backward.

Example 1:

Input: **words** = ["abc", "car", "ada", "racecar", "cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada".

Note that "racecar" is also palindromic, but it is not the first.

Example 2:

Input: words = ["notapalindrome", "racecar"]

Output: "racecar"

Explanation: The first and only string that is palindromic is "racecar".

Example 3:

Input: words = ["def", "ghi"]

Output: ""

Explanation: There are no palindromic strings, so the empty string is returned.

7. Check if the Sentence Is Pangram

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input: sentence = "thequickbrownfoxjumpsoverthelazydog"

Output: true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input: sentence = "leetcode"

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

8. Reverse Words in a String

Given a string s, reverse the order of characters in each word within a sentence while still

preserving whitespace and initial word order.

Example 1:

Input: s = "Let's take LeetCode contest"

Output: "s'teL ekat edoCteeL tsetnoc"

Example 2:

Input: s = "Mr Ding"

Output: "rM gniD"

Constraints:

$1 \leq s.length \leq 5 * 10^4$

s contains printable ASCII characters.

s does not contain any leading or trailing spaces.

There is at least one word in s.

All the words in s are separated by a single space.

9. Number of Strings That Appear as Substrings in Word

Given an array of strings patterns and a string word, return the number of strings in patterns that

exist as a substring in word.

A substring is a contiguous sequence of characters within a string.

Example 1:

Input: patterns = ["a","abc","bc","d"], word = "abc"

Output: 3

Explanation:

- "a" appears as a substring in "abc".
- "abc" appears as a substring in "abc".
- "bc" appears as a substring in "abc".
- "d" does not appear as a substring in "abc".

3 of the strings in patterns appear as a substring in word.

Example 2:

Input: patterns = ["a","b","c"], word = "aaaaabbbbb"

Output: 2

Explanation:

- "a" appears as a substring in "aaaaabbbbb".
- "b" appears as a substring in "aaaaabbbbb".
- "c" does not appear as a substring in "aaaaabbbbb".

2 of the strings in patterns appear as a substring in word.

Example 3:

Input: patterns = ["a","a","a"], word = "ab"

Output: 3

Explanation: Each of the patterns appears as a substring in word "ab".

10. Determine if String Halves Are Alike

You are given a string `s` of even length. Split this string into two halves of equal lengths, and let `a` be the first half and `b` be the second half.

Two strings are alike if they have the same number of vowels ('a', 'e', 'i', 'o', 'u', 'A', 'E', 'I', 'O', 'U').

Notice that `s` contains uppercase and lowercase letters.

Return `true` if `a` and `b` are alike. Otherwise, return `false`.

Example 1:

Input: `s = "book"`

Output: `true`

Explanation: `a = "bo"` and `b = "ok"`. `a` has 1 vowel and `b` has 1 vowel. Therefore, they are alike.

Example 2:

Input: `s = "textbook"`

Output: `false`

Explanation: `a = "text"` and `b = "book"`. `a` has 1 vowel whereas `b` has 2. Therefore, they are not alike. Notice that the vowel `o` is counted twice.

Constraints:

$2 \leq s.length \leq 1000$

`s.length` is even.

`s` consists of uppercase and lowercase letters.

11. Remove Trailing Zeros From a String

You are given a positive integer `num` represented as a string. Return the integer `num` without trailing zeros as a string.

Example 1:

Input: `num = "51230100"`

Output: `"512301"`

Explanation: Integer `"51230100"` has 2 trailing zeros, remove them and return `"512301"`.

Example 2:

Input: `num = "123"`

Output: `"123"`

Explanation: Integer `"123"` has no trailing zeros, return it as is.

Constraints:

- $1 \leq \text{num.length} \leq 1000$
- num consists of only digits
- num doesn't have any leading zeros

12. Find Common Characters

Given a string array `words`, return an array of all characters that show up in all strings within the

`words` (including duplicates). You may return the answer in any order.

Example 1:

Input: `words = ["bella", "label", "roller"]`

Output: `["e", "l", "l"]`

Example 2:

Input: `words = ["cool", "lock", "cook"]`

Output: `["c", "o"]`

Constraints:

- $1 \leq \text{words.length} \leq 100$
- $1 \leq \text{words}[i].\text{length} \leq 100$
- `words[i]` consists of lowercase English letters.

13. Delete Characters to Make Fancy String

A fancy string is a string where no three consecutive characters are equal.

Given a string `s`, delete the minimum possible number of characters from `s` to make it fancy.

Return the final string after the deletion. It can be shown that the answer will always be unique.

Example 1:

Input: `s = "leeetcode"`

Output: `"leetcode"`

Explanation:

Remove an 'e' from the first group of 'e's to create `"leetcode"`.

No three consecutive characters are equal, so return `"leetcode"`.

Example 2:

Input: `s = "aaabaaaa"`

Output: `"aabaa"`

Explanation:

Remove an 'a' from the first group of 'a's to create "aabaaaa".

Remove two 'a's from the second group of 'a's to create "aabaa".

No three consecutive characters are equal, so return "aabaa".

Example 3:

Input: s = "aab"

Output: "aab"

Explanation: No three consecutive characters are equal, so return "aab".

Constraints:

- $1 \leq s.length \leq 10^5$
- s consists only of lowercase English letters.

14. Destination City

You are given the array `paths`, where `paths[i] = [cityAi, cityBi]` means there

exists a direct path going from `cityAi` to `cityBi`. Return the destination city, that is, the city

without any path outgoing to another city.

It is guaranteed that the graph of paths forms a line without any loop, therefore, there will be exactly one destination city.

Example 1:

Input: paths = [["London","New York"],["New York","Lima"],["Lima","Sao Paulo"]]

Output: "Sao Paulo"

Explanation: Starting at "London" city you will reach "Sao Paulo" city which is the destination city.

Your trip consists of: "London" -> "New York" -> "Lima" -> "Sao Paulo".

Example 2:

Input: paths = [["B","C"],["D","B"],["C","A"]]

Output: "A"

Explanation: All possible trips are:

"D" -> "B" -> "C" -> "A".

"B" -> "C" -> "A".

"C" -> "A".

"A".

Clearly the destination city is "A".

Example 3:

Input: paths = ["A","Z"]

Output: "Z"

Constraints:

- $1 \leq \text{paths.length} \leq 100$
- $\text{paths}[i].\text{length} == 2$
- $1 \leq \text{cityAi.length}, \text{cityBi.length} \leq 10$
- $\text{cityAi} \neq \text{cityBi}$
- All strings consist of lowercase and uppercase English letters and the space character.

15. Verifying an Alien Dictionary

In an alien language, surprisingly, they also use English lowercase letters, but possibly in a different

order. The order of the alphabet is some permutation of lowercase letters.

Given a sequence of words written in the alien language, and the order of the alphabet, return true if

and only if the given words are sorted lexicographically in this alien language.

Example 1:

Input: words = ["hello","leetcode"], order = "hlabcdefghijklmnopqrstuvwxyz"

Output: true

Example 2:

Input: words = ["word","world","row"], order = "worldabcefghijklmnopqrstuvwxyz"

Output: false

Example 3:

Input: words = ["apple","app"], order = "abcdefghijklmnopqrstuvwxyz"

Output: false

Constraints:

$1 \leq \text{words.length} \leq 100$

$1 \leq \text{words}[i].\text{length} \leq 20$

$\text{order.length} == 26$

All characters in words[i] and order are English lowercase letters.

16. Check if One String Swap Can Make Strings Equal

You are given two strings `s1` and `s2` of equal length. A string swap is an operation where you choose two indices in a string (not necessarily different) and swap the characters at these indices.

Return true if it is possible to make both strings equal by performing at most one string swap on exactly one of the strings. Otherwise, return false.

Example 1:

Input: `s1 = "bank", s2 = "kanb"`

Output: true

Explanation: For example, swap the first character with the last character of `s2` to make "bank".

Example 2:

Input: `s1 = "attack", s2 = "defend"`

Output: false

Explanation: It is impossible to make them equal with one string swap.

Example 3:

Input: `s1 = "kelb", s2 = "kelb"`

Output: true

Explanation: The two strings are already equal, so no string swap operation is required.

Constraints:

$1 \leq s1.length, s2.length \leq 100$

$s1.length == s2.length$

`s1` and `s2` consist of only lowercase English letters.

17. Buddy Strings

Given two strings `s` and `goal`, return true if you can swap two letters in `s` so the result is equal to `goal`, otherwise, return false.

Swapping letters is defined as taking two indices `i` and `j` (0-indexed) such that $i \neq j$ and swapping the characters at `s[i]` and `s[j]`.

For example, swapping at indices 0 and 2 in "abcd" results in "cbad".

Example 1:

Input: `s = "ab", goal = "ba"`

Output: true

Explanation: You can swap $s[0] = 'a'$ and $s[1] = 'b'$ to get "ba", which is equal to goal.

Example 2:

Input: $s = "ab"$, $goal = "ab"$

Output: false

Explanation: The only letters you can swap are $s[0] = 'a'$ and $s[1] = 'b'$, which results in "ba" $\neq$ goal.

Example 3:

Input: $s = "aa"$, $goal = "aa"$

Output: true

Explanation: You can swap $s[0] = 'a'$ and $s[1] = 'a'$ to get "aa", which is equal to goal.

Constraints:

$1 \leq s.length, goal.length \leq 2 * 10^4$

s and goal consist of lowercase letters.

18. Word Pattern

Given a pattern and a string s, find if s follows the same pattern.

Here follow means a full match, such that there is a bijection between a letter in pattern and a non-empty word in s. Specifically:

- Each letter in pattern maps to exactly one unique word in s.
- Each unique word in s maps to exactly one letter in pattern.
- No two letters map to the same word, and no two words map to the same letter.

Example 1:

Input: pattern = "abba", s = "dog cat cat dog"

Output: true

Explanation:

The bijection can be established as:

'a' maps to "dog".

'b' maps to "cat".

Example 2:

Input: pattern = "abba", s = "dog cat cat fish"

Output: false

Example 3:

Input: pattern = "aaaa", s = "dog cat cat dog"

Output: false

Constraints:

- $1 \leq \text{pattern.length} \leq 300$
- pattern contains only lower-case English letters.
- $1 \leq \text{s.length} \leq 3000$
- s contains only lowercase English letters and spaces ' '.
- s does not contain any leading or trailing spaces.
- All the words in s are separated by a single space.

19. Second Largest Digit in a String

Given an alphanumeric string **s**, return the second largest numerical digit that appears in **s**,
or **-1** if it does not exist.

An alphanumeric string is a string consisting of lowercase English letters and digits.

Example 1:

Input: s = "dfa12321afd"

Output: 2

Explanation: The digits that appear in s are [1, 2, 3]. The second largest digit is 2.

Example 2:

Input: s = "abc1111"

Output: -1

Explanation: The digits that appear in s are [1]. There is no second largest digit.

Constraints:

- $1 \leq \text{s.length} \leq 500$
- s consists of only lowercase English letters and digits.

20. Greatest Common Divisor of Strings

For two strings **s** and **t**, we say "**t divides s**" if and only if

s = t + t + t + ... + t (t concatenated one or more times).

Given two strings **str1** and **str2**, return the largest string **x** such that **x** divides both **str1** and **str2**.

Example 1:

Input: str1 = "ABCABC", str2 = "ABC"

Output: "ABC"

Example 2:

Input: str1 = "ABABAB", str2 = "ABAB"

Output: "AB"

Example 3:

Input: str1 = "LEET", str2 = "CODE"

Output: ""

Constraints:

- $1 \leq \text{str1.length}, \text{str2.length} \leq 1000$
- str1 and str2 consist of English uppercase letters

21. Find Unique Binary String**Problem Description**

You are given an array of strings `nums` containing n **unique binary strings**, each of length n .

Your task is to return a **binary string of length n** that **does not appear in `nums`**.

If there are multiple valid answers, **you may return any of them**.

Examples**Example 1:**

Input: `nums = ["01", "10"]`

Output: "11"

Explanation:

The binary string "11" does not appear in the input.

Other valid answers include "00".

Example 2:

Input: `nums = ["00", "01"]`

Output: "11"

Explanation:

The binary string "11" is not in the input list.

Other valid answers include "10".

Example 3:

Input: `nums = ["111", "011", "001"]`

Output: "101"

Explanation:

The binary string "101" is not in the input list.

Other valid answers include "000", "010", "100", "110", etc.

Constraints

- $n == \text{nums.length}$
- $1 \leq n \leq 16$
- $\text{nums}[i].\text{length} == n$
- $\text{nums}[i]$ consists only of '0' or '1' characters

- All strings in `nums` are **unique**

22. Adding Spaces to a String

Problem

You are given a 0-indexed string `s` and a 0-indexed integer array `spaces` that describes the

indices in the original string where spaces will be added. Each space should be inserted before the

character at the given index.

For example, given `s = "EnjoyYourCoffee"` and `spaces = [5, 9]`, we place

spaces before 'Y' and 'C', which are at indices 5 and 9 respectively. Thus, we obtain "Enjoy

Your Coffee".

Return the modified string after the spaces have been added.

Example 1:

Input:

`s = "LeetcodeHelpsMeLearn"`, `spaces = [8,13,15]`

Output:

"Leetcode Helps Me Learn"

Explanation:

The indices 8, 13, and 15 correspond to the underlined characters in "LeetcodeHelpsMeLearn".

We then place spaces before those characters.

Example 2:

Input:

`s = "icodeinpython"`, `spaces = [1,5,7,9]`

Output:

"i code in py thon"

Explanation:

The indices 1, 5, 7, and 9 correspond to the underlined characters in "icodeinpython".

We then place spaces before those characters.

Example 3:

Input:

Output:

" s p a c i n g"

Explanation:

```
s = "spacing", spaces = [0,1,2,3,4,5,6]
```

We are also able to place spaces before the first character of the string.

Constraints:

- $1 \leq s.length \leq 3 * 10^5$
- `s` consists only of lowercase and uppercase English letters.
- $1 \leq spaces.length \leq 3 * 10^5$
- $0 \leq spaces[i] \leq s.length - 1$
- All the values of `spaces` are strictly increasing.

23. Group Anagrams

Problem

order.

Given an array of strings `strs`, group the anagrams together. You can return the answer in any

Example 1:

Input:

```
strs = ["eat", "tea", "tan", "ate", "nat", "bat"]
```

Output:

```
[["bat"], ["nat", "tan"], ["ate", "eat", "tea"]]
```

Explanation:

- There is no string in `strs` that can be rearranged to form "bat".
- The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.
- The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.

Example 2:

Input:

```
strs = [ " " ]
```

Output:

```
[ [ " " ] ]
```

Example 3:

Input:

```
strs = [ "a" ]
```

Output:

```
[ [ "a" ] ]
```

Constraints:

- `1 <= strs.length <= 10^4`
- `0 <= strs[i].length <= 100`
- `strs[i]` consists of lowercase English letters.

24. Find Resultant Array After Removing Anagrams

Problem

You are given a 0-indexed string array `words`, where `words[i]` consists of lowercase English letters.

In one operation, select any index `i` such that `0 < i < words.length` and `words[i - 1]` and `words[i]` are anagrams, and delete `words[i]` from `words`. Keep

performing this operation as long as you can select an index that satisfies the conditions.

Return `words` after performing all operations. It can be shown that selecting the

indices for each operation in any arbitrary order will lead to the same result.

An Anagram is a word or phrase formed by rearranging the letters of a different word

or phrase using all the original letters exactly once. For example, "dacb" is an anagram of "abdc".

Example 1:

Input:

`words = ["abba", "baba", "bbaa", "cd", "cd"]`

Output:

`["abba", "cd"]`

Explanation:

One of the ways we can obtain the resultant array is by using the following operations:

- Since `words[2] = "bbaa"` and `words[1] = "baba"` are anagrams, we choose

index 2 and delete `words[2]`.

Now `words = ["abba", "baba", "cd", "cd"]`.

- Since `words[1] = "baba"` and `words[0] = "abba"` are anagrams, we choose index 1 and delete `words[1]`.
Now `words = ["abba", "cd", "cd"]`.
- Since `words[2] = "cd"` and `words[1] = "cd"` are anagrams, we choose index 2 and delete `words[2]`.
Now `words = ["abba", "cd"]`.
We can no longer perform any operations, so `["abba", "cd"]` is the final answer.

Example 2:

Input:

`words = ["a", "b", "c", "d", "e"]`

Output:

`["a", "b", "c", "d", "e"]`

Explanation:

No two adjacent strings in `words` are anagrams of each other, so no operations are performed.

Constraints:

- `1 <= words.length <= 100`
- `1 <= words[i].length <= 10`
- `words[i]` consists of lowercase English letters.

25. Check If a String Can Break Another String

Problem

Given two strings: `s1` and `s2` with the same size, check if some permutation of string `s1` can

break some permutation of string `s2` or vice-versa. In other words `s2` can break `s1` or vice-versa.

A string `x` can break string `y` (both of size `n`) if `x[i]` between 0 and `n-1`.

`>= y[i]` (in alphabetical order) for all `i`

Example 1:

Input:

`s1 = "abc", s2 = "xya"`

Output:

`true`

Explanation:

"ayx" is a permutation of `s2` permutation of `s1 = "abc"`.

= "xya" which can break to string "abc" which is a

Example 2:

Input:

`s1 = "abe", s2 = "acd"`

Output:

`false`

Explanation:

All permutations for `s1 = "abe"` are: `"abe", "aeb", "bae", "bea", "eab"` and

`"eba"`, and all permutations for `s2 = "acd"` are: `"acd", "adc", "cad", "cda",`

`"dac"` and `"dca"`. However, there is not any permutation from `s1` which can break some

permutation from `s2` and vice-versa.

Example 3:

Input:

`s1 = "leetcode", s2 = "interview"`

Output:

`true`

Constraints:

- `s1.length == n`
- `s2.length == n`
- `1 <= n <= 105`
- All strings consist of lowercase English letters.

26. Count Vowel Strings in Ranges**Problem**

You are given a 0-indexed array of strings `words` and a 2D array of integers `queries`.

Each query `queries[i] = [li, ri]` asks us to find the number of strings present at the

indices ranging from `li` to `ri` (both inclusive) of `words` that start and end with a vowel.

Return an array `ans` of size `queries.length`, where `ans[i]` is the answer to the `i`th query.

Note that the vowel letters are `'a', 'e', 'i', 'o', and 'u'`.

Example 1:

Input:

```
words = ["aba", "bcb", "ece", "aa", "e"], queries = [[0,2],
[1,4],
[1,1]]
```

Output:

```
[2,3,0]
```

Explanation:

The strings starting and ending with a vowel are "aba", "ece", "aa" and "e".

The answer to the query [0,2] is 2 (strings "aba" and "ece").

The answer to the query [1,4] is 3 (strings "ece", "aa", "e").

The answer to the query [1,1] is 0.

We return [2,3,0].

Example 2:

Input:

```
words = ["a", "e", "i"], queries = [[0,2],[0,1],[2,2]]
```

Output:

```
[3,2,1]
```

Explanation:

Every string satisfies the conditions, so we return [3,2,1].

Constraints:

- $1 \leq \text{words.length} \leq 10^5$
- $1 \leq \text{words}[i].\text{length} \leq 40$
- $\text{words}[i]$ consists only of lowercase English letters.
- $\sum(\text{words}[i].\text{length}) \leq 3 * 10^5$
- $1 \leq \text{queries.length} \leq 10^5$
- $0 \leq l_i \leq r_i < \text{words.length}$

27. Minimum Deletions to Make Character Frequencies Unique

Problem

A string **s** is called **good** if there are no two different characters in **s** that have the same frequency.

Given a string **s**, return the minimum number of characters you need to delete to make **s** good.

The frequency of a character in a string is the number of times it appears in the string.

For example,

in the string "aab", the frequency of 'a' is 2, while the frequency of 'b' is 1.

Example 1:

Input:

```
s = "aab"
```

Output:

0

Explanation:

s is already good.

Example 2:

Input:

s = "aaabbbcc"

Output:

2

Explanation:

You can delete two 'b' s resulting in the good string "aaabcc".

Another way is to delete one 'b' and one 'c' resulting in the good string "aaabbc".

Example 3:

Input:

s = "ceabaacb"

Output:

2

Explanation:

You can delete both 'c' s resulting in the good string "eabaab".

Note that we only care about characters that are still in the string at the end (i.e. frequency of 0 is ignored).

Constraints:

- $1 \leq s.length \leq 10^5$
- s contains only lowercase English letters.

28. Next Permutation (Recursive)

Description: Given a number in string format, return the next lexicographic permutation.

Input: s: str

Output: str

Constraints: $1 \leq \text{len}(s) \leq 8$

Example:

Input: s = "123"

Output: "132"

29. Word Break with Permutations

Description: Return all permutations of valid word breaks using a given dictionary.

Input: s: str, wordDict: List[str]

Output: List[str]

Constraints: $1 \leq \text{len}(s) \leq 20$

Example:

Input: s = "catsanddog", wordDict = ["cat", "cats", "and", "sand", "dog"]

Output: ["cats and dog", "cat sand dog"]

30. Generate Balanced Parentheses

Description: Generate all combinations of well-formed n pairs of parentheses.

Input: n: int

Output: List[str]

Constraints: $1 \leq n \leq 10$

Example:

Input: n = 3

Output: ["((()))", "(()())", "()(())", "())()", "())()"]

31. Binary Strings Without Consecutive 1s

Description: Generate all binary strings of length n with no consecutive 1s.

Input: n: int

Output: List[str]

Constraints: $1 \leq n \leq 10$

Example:

Input: n = 3

Output: ["000", "001", "010", "100", "101"]

32. Palindrome Partitioning

Description: Given a string, return all possible palindrome partitionings.

Input: s: str

Output: List[List[str]]

Constraints: $1 \leq \text{len}(s) \leq 15$

Example:

Input: $s = \text{"aab"}$

Output: $[[\text{"a"}, \text{"a"}, \text{"b"}], [\text{"aa"}, \text{"b"}]]$

33. Determine if Two Strings Are Close

Two strings are considered **close** if you can attain one from the other using the following operations:

- Operation 1: Swap any two **existing** characters.
- For example, $\text{abcde} \rightarrow \text{aecdb}$
- Operation 2: Transform **every** occurrence of one **existing** character into another **existing** character, and do the same with the other character.
- For example, $\text{aacabb} \rightarrow \text{bbcbaa}$ (all a's turn into b's, and all b's turn into a's)

You can use the operations on either string as many times as necessary. Given two strings, word1 and word2, return *true if word1 and word2 are close, and false otherwise*.

Example 1:

Input: word1 = "abc", word2 = "bca"

Output: true

Explanation: You can attain word2 from word1 in 2 operations.

Apply Operation 1: "abc" $\rightarrow$ "acb"

Apply Operation 1: "acb" $\rightarrow$ "bca"

Example 2:

Input: word1 = "a", word2 = "aa"

Output: false

Explanation: It is impossible to attain word2 from word1, or vice versa, in any number of operations.

Example 3:

Input: word1 = "cabbba", word2 = "abbccc"

Output: true

Explanation: You can attain word2 from word1 in 3 operations.

Apply Operation 1: "cabbba" $\rightarrow$ "caabbb"

Apply Operation 2: "caabbb" $\rightarrow$ "baaccc"

Apply Operation 2: "baaccc" $\rightarrow$ "abbccc"

Constraints:

- $1 \leq \text{word1.length}, \text{word2.length} \leq 105$
- word1 and word2 contain only lowercase English letters.

34. Find All Anagrams in a String

Given two strings *s* and *p*, return an array of all the start indices of *p*'s anagrams in *s*. You may return the answer in **any order**.

Example 1:

Input: *s* = "cbaebabacd", *p* = "abc"

Output: [0,6]

Explanation:

The substring with start index = 0 is "cba", which is an anagram of "abc".

The substring with start index = 6 is "bac", which is an anagram of "abc".

Example 2:

Input: *s* = "abab", *p* = "ab"

Output: [0,1,2]

Explanation:

The substring with start index = 0 is "ab", which is an anagram of "ab".

The substring with start index = 1 is "ba", which is an anagram of "ab".

The substring with start index = 2 is "ab", which is an anagram of "ab".

Constraints:

- $1 \leq s.length, p.length \leq 3 * 10^4$
- *s* and *p* consist of lowercase English letters.

35. Second Largest Digit in a String

Given an alphanumeric string *s*, return the **second largest** numerical digit that appears in *s*, or -1 if it does not exist.

An **alphanumeric** string is a string consisting of lowercase English letters and digits.

Example 1:

Input: *s* = "dfa12321afd"

Output: 2

digit is 2.

Example 2:

Input: *s* = "abc1111"

Output: -1

largest digit.

Constraints:

- $1 \leq s.length \leq 500$
- *s* consists of only lowercase English letters and digits.

36. Word Pattern

Given a pattern and a string s, find if s follows the same pattern.

Here **follow** means a full match, such that there is a bijection between a letter in pattern and a **non-empty** word in s. Specifically:

- Each letter in pattern maps to **exactly** one unique word in s.
- Each unique word in s maps to **exactly** one letter in pattern.
- No two letters map to the same word, and no two words map to the same letter.

Example 1:

Input: pattern = "abba", s = "dog cat cat dog"

Output: true

Explanation:

The bijection can be established as:

- 'a' maps to "dog".
- 'b' maps to "cat".

Example 2:

Input: pattern = "abba", s = "dog cat cat fish"

Output: false

Example 3:

Input: pattern = "aaaa", s = "dog cat cat dog"

Output: false

Constraints:

- $1 \leq \text{pattern.length} \leq 300$
- pattern contains only lower-case English letters.
- $1 \leq \text{s.length} \leq 3000$
- s contains only lowercase English letters and spaces ' '.
- s **does not contain** any leading or trailing spaces.
- All the words in s are separated by a **single space**.

37. Buddy Strings

Given two strings s and goal, return true if you can swap two letters in s so the result is equal to goal, otherwise, return false.

Swapping letters is defined as taking two indices i and j (0-indexed) such that $i \neq j$ and swapping the characters at $s[i]$ and $s[j]$.

- For example, swapping at indices 0 and 2 in "abcd" results in "cbad".

Example 1:

Input: s = "ab", goal = "ba"

Output: true

Explanation: You can swap $s[0] = 'a'$ and $s[1] = 'b'$ to get "ba", which is equal to goal.

Example 2:

Input: `s = "ab", goal = "ab"`

Output: false

Explanation: The only letters you can swap are `s[0] = 'a'` and `s[1] = 'b'`, which results in `"ba" != goal`.

Example 3:

Input: `s = "aa", goal = "aa"`

Output: true

Explanation: You can swap `s[0] = 'a'` and `s[1] = 'a'` to get `"aa"`, which is equal to goal.

Constraints:

- $1 \leq s.length, goal.length \leq 2 * 10^4$
- `s` and `goal` consist of lowercase letters.

38. Minimum Number of Steps to Make Two Strings Anagram II

You are given two strings `s` and `t`. In one step, you can append **any character** to either `s` or `t`.

Return the minimum number of steps to make `s` and `t` **anagrams** of each other.

An **anagram** of a string is a string that contains the same characters with a different (or the same) ordering.

Example 1:

Input: `s = "leetcode", t = "coats"`

Output: 7

Explanation:

- In 2 steps, we can append the letters in `"as"` onto `s = "leetcode"`, forming `s = "leetcodeas"`.

- In 5 steps, we can append the letters in `"leede"` onto `t = "coats"`, forming `t = "coatsleede"`.

`"leetcodeas"` and `"coatsleede"` are now anagrams of each other.

We used a total of $2 + 5 = 7$ steps.

It can be shown that there is no way to make them anagrams of each other with less than 7 steps.

Example 2:

Input: `s = "night", t = "thing"`

Output: 0

Explanation: The given strings are already anagrams of each other. Thus, we do not need any further steps.

Constraints:

- $1 \leq s.length, t.length \leq 2 * 10^5$
- `s` and `t` consist of lowercase English letters.

39. Delete Characters to Make Fancy String

A **fancy string** is a string where no **three consecutive** characters are equal. Given a string *s*, delete the **minimum** possible number of characters from *s* to make it **fancy**.

Return *the final string after the deletion*. It can be shown that the answer will always be **unique**.

Example 1:

Input: *s* = "leeetcode"

Output: "leetcode"

Explanation:

Remove an 'e' from the first group of 'e's to create "leetcode".

No three consecutive characters are equal, so return "leetcode".

Example 2:

Input: *s* = "aaabaaaa"

Output: "aabaa"

Explanation:

Remove an 'a' from the first group of 'a's to create "aabaaaa".

Remove two 'a's from the second group of 'a's to create "aabaa".

No three consecutive characters are equal, so return "aabaa".

Example 3:

Input: *s* = "aab"

Output: "aab"

Explanation: No three consecutive characters are equal, so return "aab".

Constraints:

- $1 \leq s.length \leq 105$
- *s* consists only of lowercase English letters.

40. Greatest English Letter in Upper and Lower Case

Given a string of English letters *s*, return the **greatest** English letter which occurs as **both** a lowercase and uppercase letter in *s*. The returned letter should be in **uppercase**. If no such letter exists, return an empty string.

An English letter *b* is **greater** than another letter *a* if *b* appears **after** *a* in the English alphabet.

Example 1:

Input: *s* = "lEeTcOdE"

Output: "E"

Explanation:

The letter 'E' is the only letter to appear in both lower and upper case.

Example 2:

Input: *s* = "arRAzFif"

Output: "R"

Explanation:

The letter 'R' is the greatest letter to appear in both lower and upper case. Note that 'A' and 'F' also appear in both lower and upper case, but 'R' is greater than 'F' or 'A'.

Example 3:

Input: s = "AbCdEfGhIjK"

Output: ""

Explanation:

There is no letter that appears in both lower and upper case.

Constraints:

- $1 \leq s.length \leq 1000$
- s consists of lowercase and uppercase English letters.

41. Length of Last Word

Given a string s consisting of words and spaces, return *the length of the last word in the string*.

A **word** is a maximal substring consisting of non-space characters only.

Example 1:

Input: s = "Hello World"

Output: 5

Explanation: The last word is "World" with length 5.

Example 2:

Input: s = " fly me to the moon "

Output: 4

Explanation: The last word is "moon" with length 4.

Example 3:

Input: s = "luffy is still joyboy"

Output: 6

Explanation: The last word is "joyboy" with length 6.

Constraints:

- $1 \leq s.length \leq 104$
- s consists of only English letters and spaces ' '.
- There will be at least one word in s.

42. Word Pattern

Given a pattern and a string s, find if s follows the same pattern.

Here **follow** means a full match, such that there is a bijection between a letter in pattern and a **non-empty** word in s. Specifically:

- Each letter in pattern maps to **exactly** one unique word in s.
- Each unique word in s maps to **exactly** one letter in pattern.
- No two letters map to the same word, and no two words map to the same letter.

Example 1:

Input: pattern = "abba", s = "dog cat cat dog"

Output: true

Explanation:

The bijection can be established as:

- 'a' maps to "dog".
- 'b' maps to "cat".

Example 2:

Input: pattern = "abba", s = "dog cat cat fish"

Output: false

Example 3:

Input: pattern = "aaaa", s = "dog cat cat dog"

Output: false

Constraints:

- $1 \leq \text{pattern.length} \leq 300$
- pattern contains only lower-case English letters.
- $1 \leq \text{s.length} \leq 3000$
- s contains only lowercase English letters and spaces ' '.
- s **does not contain** any leading or trailing spaces.
- All the words in s are separated by a **single space**.

43. Bracket Validation with Priority Levels

Problem: Given an expression containing brackets of different types, validate it ensuring higher-priority brackets are nested inside lower-priority ones.

Input: String expression like [{ () }] **Output:** true or false

Constraints:

- Priorities: { > [> (
- Length $\leq 10^4$

Example: Input: { [()] } Output: true

Input: [{ () }] Output: false

44. Parenthesis Depth Counter

Problem: Return the maximum depth of nested parentheses in a given string.

Input: String s **Output:** Integer depth

Constraints:

- s contains valid parentheses
- Length $\leq 10^4$

Example: Input: ((a+ (b)) + (c+d)) Output: 3

45. Expression Simplifier

Problem: Simplify arithmetic expressions by removing parentheses and flattening signs.

Input: String s like a- (b+c- (d+e)) -f **Output:** Flattened string

Constraints:

- Only lowercase letters and +/-()`
- Length $\leq 10^4$

Example: Input: a- (b+c- (d+e)) -f Output: a-b-c+d+e-f

46. Remove Duplicate Letters Lexicographically Smallest

Problem: Remove duplicate letters to get smallest lexicographical order.

Input: String s **Output:** Modified string

Constraints:

- Only lowercase letters
- Length $\leq 10^4$

Example: Input: bcabc Output: abc

47. Shortest Unique Substring Containing All Characters

Problem: Return the smallest substring containing all unique characters in string.

Input: String s **Output:** Shortest substring

Constraints:

- Only lowercase letters
- Length $\leq 10^4$

Example: Input: aabcbcbdbca Output: dbca

48. Rearrange String to Avoid Adjacent Duplicates

Problem: Rearrange to avoid adjacent identical characters.

Input: String s **Output:** Rearranged string or ""

Constraints:

- Only lowercase letters
- Length $\leq 10^5$

Example: Input: aaabc Output: abaca

49. Find and Replace Pattern Matching

Problem: Return all words matching a given pattern of letter positions.

Input: String[] words, String pattern **Output:** List of words

Constraints:

- Length of pattern ≤ 100
- $1 \leq \text{words.length} \leq 1000$

Example: Input: words = ["mee", "aqq", "dkd", "ccc"], pattern = "abb" Output: ["mee", "aqq"]

50. Minimum Insertions to Make String Palindrome

Problem: Find the minimum number of insertions required to make a string palindrome.

Input: String s **Output:** int (min insertions)

Constraints:

- Only lowercase letters
- Length ≤ 500

Example: Input: abcaa Output: 1

51. Multiply Strings Without Integer Conversion

Problem: Multiply two large numbers represented as strings.

Input: String num1, String num2 **Output:** Product as string

Constraints:

- No built-in bigint or integer parsing
- Length ≤ 500

Example: Input: "123", "456" Output: "56088"

52. Longest Substring with Each Char at Least K Times

Problem: Return the length of longest substring where each character appears at least

k times.

Input: String s, int k **Output:** int length

Constraints:

- $1 \leq s.length \leq 10^4$
- $1 \leq k \leq 100$

Example: Input: s = "aaabb", k = 3 Output: 3

53. Shortest Distance to Target String in a Circular Array

You are given a 0-indexed circular string array words and a string target. A circular

array means that the array's end connects to the array's beginning.

Formally, the next element of words[i] is words[(i + 1) % n] and the previous

element of words[i] is words[(i - 1 + n) % n], where n is the length of words.

Starting from startIndex, you can move to either the next word or the previous word with 1 step at a time.

Return the shortest distance needed to reach the string target. If the string target does not exist in words, return -1.

Example 1:

Input: words = ["hello","i","am","leetcode","hello"], target = "hello", startIndex = 1

Output: 1

Explanation:

We start from index 1 and can reach "hello" by

- moving 3 units to the right to reach index 4.
- moving 2 units to the left to reach index 4.
- moving 4 units to the right to reach index 0.
- moving 1 unit to the left to reach index 0.

The shortest distance to reach "hello" is 1.

Example 2:

Input: words = ["a","b","leetcode"], target = "leetcode", startIndex = 0

Output: 1

Explanation:

We start from index 0 and can reach "leetcode" by

- moving 2 units to the right to reach index 2.
- moving 1 unit to the left to reach index 2.

The shortest distance to reach "leetcode" is 1.

Example 3:

Input: words = ["i","eat","leetcode"], target = "ate", startIndex = 0

Output: -1

Explanation:

Since "ate" does not exist in words, we return -1.

Constraints:

- $1 \leq \text{words.length} \leq 100$
- $1 \leq \text{words}[i].\text{length} \leq 100$
- words[i] and target consist of only lowercase English letters.
- $0 \leq \text{startIndex} < \text{words.length}$